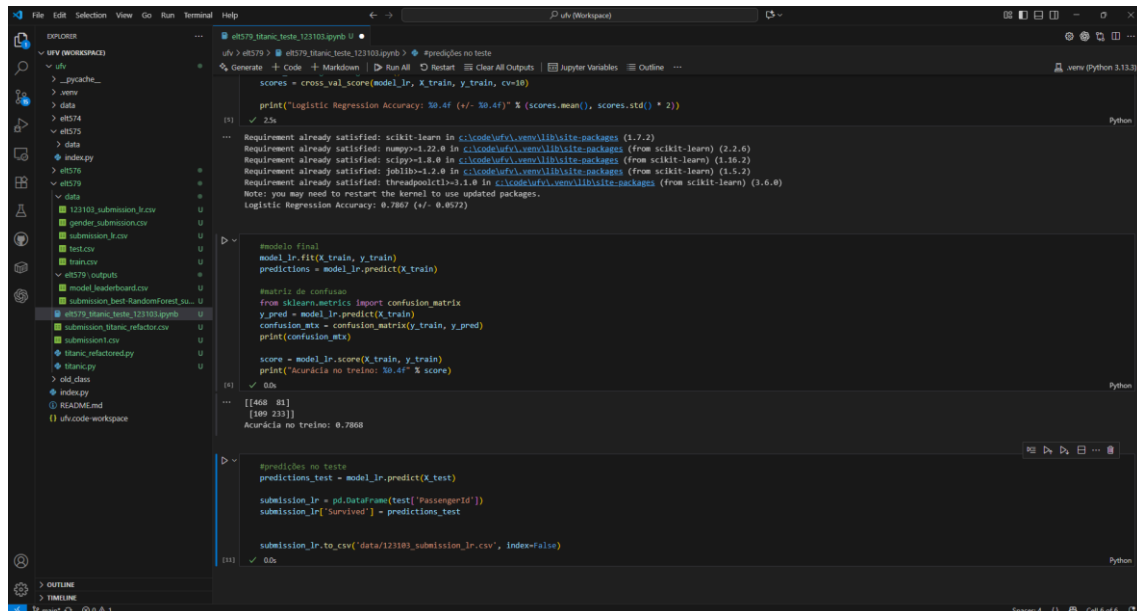


ELT579 – Relatório 1 – 123103 – Elmer A. M. Rodrigues

Logistic Regression

Para realizar o trabalho, utilizei o VSCode que eu já tenho mais familiaridade. Consegui instalar o Jupyter no VSCode e foi interessante conseguir executando blocos localmente.

Segui o passo a passo da primeira aula e consegui o mesmo resultado, conforme esperado.



```
ufv > elt579 > elt579_titanic_teste_123103.ipynb > #predições no teste
scores = cross_val_score(model_lr, x_train, y_train, cv=10)

print("Logistic Regression Accuracy: %.4f (+/- %.4f) % (scores.mean(), scores.std() * 2))

(1) ✓ 25s

...
Requirement already satisfied: scikit-learn in c:\code\ufv\venv\lib\site-packages (1.7.2)
Requirement already satisfied: numpy<1.22.0 in c:\code\ufv\venv\lib\site-packages (from scikit-learn) (2.2.0)
Requirement already satisfied: scipy<1.8.0 in c:\code\ufv\venv\lib\site-packages (from scikit-learn) (1.10.2)
Requirement already satisfied: joblib<1.2.0 in c:\code\ufv\venv\lib\site-packages (from scikit-learn) (1.5.2)
Requirement already satisfied: threadpoolctl<3.1.0 in c:\code\ufv\venv\lib\site-packages (from scikit-learn) (3.0.0)
Note: you may need to restart the kernel to use updated packages.
Logistic Regression Accuracy: 0.7887 (+/- 0.0572)

#modelo final
model_lr.fit(x_train, y_train)
predictions = model_lr.predict(x_train)

#matriz de confusão
from sklearn.metrics import confusion_matrix
y_pred = model_lr.predict(x_train)
confusion_mtx = confusion_matrix(y_train, y_pred)
print(confusion_mtx)

score = model_lr.score(x_train, y_train)
print("Acurácia no treino: %.4f % score")

(1) ✓ 0s

...
[[468  81]
 [189 233]]
Acurácia no treino: 0.7888

#predições no teste
predictions_test = model_lr.predict(x_test)

submission_lr = pd.DataFrame(test['PassengerId'],
                             submission_lr['Survived'] = predictions_test

submission_lr.to_csv('data/123103_submission_lr.csv', index=False)

(1) ✓ 0s
```

Resultado no Kaggle:

≡

kaggle

+

Create

🏠

Home

🏆

Competitions

📁

Datasets

👤

Models

📊

Benchmarks

<>

Code

📄

Discussions

🎓

Learn

▼

More

🔍

Search

Titanic - Machine Learning from Disaster

Submit Prediction

...

Overview

Data

Code

Models

Discussion

Leaderboard

Rules

Team

Submissions

Submission and Description

Public Score ⓘ

✓

123103_submission_lr.csv

Complete · 10m ago

0.76555

✓

submission_best-RandomForest_sub2.csv

Complete · 2d ago

0.76555

✓

submission_titanic_refactor.csv

Complete · 2d ago · Refatorado - sub1

0.75119

Observação: Eu havia subido um teste de Random Forest gerado a partir do Codex para testar e não teve um desempenho maior do que fazendo o Logistic Regression.

Logistic Regression com Standard Scaler

```
#Padronizar dados
from sklearn.preprocessing import StandardScaler
scaler = StandardScaler()
X_train_sc = scaler.fit_transform(X_train)
X_test_sc = scaler.transform(X_test)

#Modelo e validação

#logistic regression
from sklearn.linear_model import LogisticRegression
from sklearn.model_selection import cross_val_score

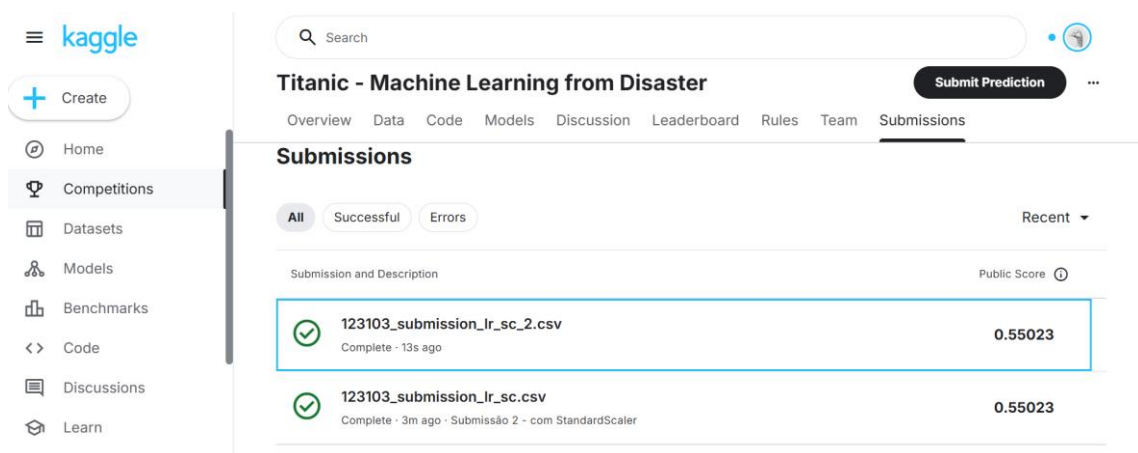
# print(X_train.head())
# print(y_train.head())

model_lr = LogisticRegression(max_iter=1000, random_state=0)

score = cross_val_score(model_lr, X_train_sc, y_train, cv=10)

print("Logistic Regression Accuracy: %.4f" % (score.mean()))
# Logistic Regression Accuracy: 0.7898 - USANDO MEDIANA PARA AGE E FARE
# Logistic Regression Accuracy: 0.7898 - USANDO MEDIA PARA AGE E FARE
```

Meu score piorou muito com a padronização de dados, caindo para 0.55



Titanic - Machine Learning from Disaster		Submit Prediction
Overview	Data	Code
Submissions		
All	Successful	Errors
Submission and Description		Public Score
123103_submission_lr_sc_2.csv Complete · 13s ago		0.55023
123103_submission_lr_sc.csv Complete · 3m ago · Submissão 2 - com StandardScaler		0.55023

Adicionei a feature “isChild” e melhorou significativamente o resultado conforme abaixo:

```
#criar features
def criar_features(x):
    subsFemale = {'male': 0, 'female': 1}
    x['isFemale'] = x['Sex'].replace(subsFemale)

    x['Age'].fillna(x['Age'].mean(), inplace=True)
    x['Fare'].fillna(x['Fare'].mean(), inplace=True)
    x['Embarked'].fillna('S', inplace=True)

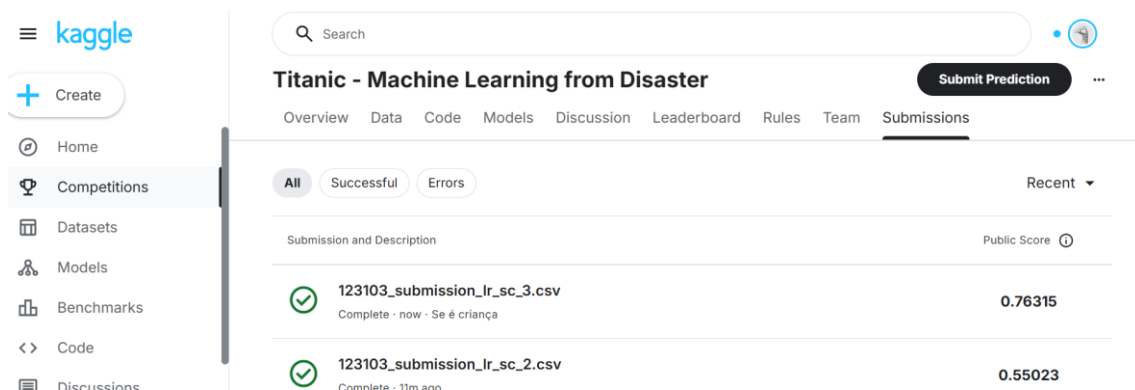
    subsPort = {'S': 0, 'C': 1, 'Q': 2}
    x['Port'] = x['Embarked'].replace(subsPort)

    x['isChild'] = np.where(x['Age'] < 12, 1, 0)

    return x
```

109] ✓ 0.0s

Resultado no Kaggle:



The screenshot shows the Kaggle interface for the 'Titanic - Machine Learning from Disaster' competition. The left sidebar contains navigation links: Home, Competitions, Datasets, Models, Benchmarks, Code, and Discussion. The main area displays the 'Submissions' tab with a table of submission results. The table has columns for 'Submission and Description' and 'Public Score'. Two submissions are listed, both marked as 'Complete' with a green checkmark icon. The first submission, '123103_submission_lr_sc_3.csv', has a public score of 0.76315. The second submission, '123103_submission_lr_sc_2.csv', has a public score of 0.55023.

Submission and Description	Public Score
✓ 123103_submission_lr_sc_3.csv Complete · now · Se é criança	0.76315
✓ 123103_submission_lr_sc_2.csv Complete · 11m ago	0.55023

Criando features

```
#criar features
def criar_features(x):
    subsFemale = {'male': 0, 'female': 1}
    x['isFemale'] = x['Sex'].replace(subsFemale)

    x['Age'].fillna(x['Age'].mean(), inplace=True)
    x['Fare'].fillna(x['Fare'].mean(), inplace=True)
    x['Embarked'].fillna('S', inplace=True)

    subsPort = {'S': 0, 'C': 1, 'Q': 2}
    x['Port'] = x['Embarked'].replace(subsPort)

    x['isChild'] = np.where(x['Age'] < 12, 1, 0)

    # Criando feature de quem pagou mais caro sobreviveu
    x['FareBin'] = pd.qcut(x['Fare'], 4, labels=False)

    #criando feature de quem embarcou no porto C sobreviveu
    x['PortC'] = np.where(x['Port'] == 1, 1, 0)

    #criando feature de quem nao tinha parente sobreviveu
    x['NoRelatives'] = np.where((x['SibSp'] + x['Parch']) == 0, 1, 0)

    return x
```

✓ 0.0s

Depois de criar algumas features observando os dados, consegui melhorar o resultado do modelo.

☰

kaggle

+

Create

🏠

Home

🏆

Competitions

📊

Datasets

👤

Models

📈

Benchmarks

<>

Code

📄

Discussions

🎓

Learn

▼

More

🔍

Search

Titanic - Machine Learning from Disaster

Submit Prediction

...

Overview

Data

Code

Models

Discussion

Leaderboard

Rules

Team

Submissions

All

Successful

Errors

Recent

Submission and Description

Public Score

✓

123103_submission_lr_sc_novas_features_4.csv

Complete · now

0.77511

✓

123103_submission_lr_sc_3.csv

Complete · 16m ago · Se é criança

0.76315

✓

123103_submission_lr_sc_2.csv

Complete · 27m ago

0.55023

Gaussian Naive e KNN

O Gaussian Naive Bayes teve uma acurácia de: 0.7689 e não submeti ao teste.

Já o KNN teve uma melhora do resultado de teste e foi submetido no Kaggle.

```
# Gaussian Naive Bayes
from sklearn.naive_bayes import GaussianNB

model_nb = GaussianNB()
model_nb.fit(X_train, y_train)

score = cross_val_score(model_nb, X_train, y_train, cv=10)
print("Gaussian Naive Bayes Accuracy: %0.4f" % (np.mean(score)))

✓ 0.0s

Gaussian Naive Bayes Accuracy: 0.7689

# KNN para classificação
from sklearn.neighbors import KNeighborsClassifier

model_knn = KNeighborsClassifier(n_neighbors=5)
model_knn.fit(X_train, y_train)

score = cross_val_score(model_knn, X_train, y_train, cv=10)
print("KNN Accuracy: %0.4f" % (np.mean(score)))

✓ 0.0s

KNN Accuracy: 0.8025

# Modelo final
model_knn.fit(X_train, y_train)
y_pred = model_knn.predict(X_train)
predictions_test = model_knn.predict(X_test)

score = model_knn.score(X_train, y_train)
print("Score: %0.4f" % (score))

submission = pd.DataFrame(test['PassengerId'])
submission['Survived'] = predictions_test

submission.to_csv('data/123103_submission_knn.csv', index=False)

✓ 0.0s

Score: 0.8462
```

Porém o resultado ainda ficou pior do que as features ajustadas anteriormente.

Submission and Description	Public Score
✓ 123103_submission_knn.csv Complete · 3m ago · Modelo KNN com minhas features	0.75837
✓ 123103_submission_lr_sc_novas_features_4.csv Complete · 14m ago	0.77511
✓ 123103_submission_lr_sc_3.csv Complete · 30m ago · Se é criança	0.76315
✓ 123103_submission_lr_sc_2.csv Complete · 41m ago	0.55023

SVM

O modelo SVM apresentou melhora no resultado sem muitos parâmetros

```
#SVM para classificação
from sklearn.svm import SVC
model_svm = SVC(C=1, gamma='auto')
model_svm.fit(X_train, y_train)

score = cross_val_score(model_svm, X_train, y_train, cv=10)
print("SVM Accuracy: %0.4f" % (np.mean(score)))
```

[221] ✓ 0.1s

... SVM Accuracy: 0.8216

O resultado do Kaggle foi o melhor até agora:

Submission and Description		Public Score
✓ 123103_submission_model_svm.csv	Complete · 1m ago · model_svm	0.77990
✓ 123103_submission_knn.csv	Complete · 9m ago · Modelo KNN com minhas features	0.75837
✓ 123103_submission_lr_sc_novas_features_4.csv	Complete · 19m ago	0.77511

Decision Tree

O modelo de Decision Tree não apresentou um resultado melhor que os anteriores, ficando com a acurácia de 0.8058 e não foi postado no Kaggle.

```
# Decision Tree
from sklearn.tree import DecisionTreeClassifier
model_tree = DecisionTreeClassifier(criterion='entropy', max_depth=3, min_samples_leaf=1, min_samples_split=5, random_state=0)
model_tree.fit(X_train, y_train)

score = cross_val_score(model_tree, X_train, y_train, cv=10)
print("Decision Tree Accuracy: %0.4f" % (np.mean(score)))
```

✓ 0.0s

Decision Tree Accuracy: 0.8058

Random Forest

O Random Forest apresentou uma acurácia maior do que outros modelos e foi submetido no Kaggle obtendo o melhor resultado dos testes.

```
# Random Forest
from sklearn.ensemble import RandomForestClassifier
model_forest = RandomForestClassifier(n_estimators=100, criterion='entropy', max_depth=5, min_samples_leaf=1, min_samples_split=3, random_state=0)
model_forest.fit(X_train, y_train)

score = cross_val_score(model_forest, X_train, y_train, cv=10)
print("Random Forest Accuracy: %0.4f" % (np.mean(score)))
```

✓ 0.9s

Random Forest Accuracy: 0.8227

Resultado do Kaggle:

≡

kaggle

+

Create

🏠

Home

🏆

Competitions

📁

Datasets

🔗

Models

📊

Benchmarks

⏏

Code

💬

Discussions

🎓

Learn

▼

More

🔍

Search

Titanic - Machine Learning from Disaster

Submit Prediction

...

Overview

Data

Code

Models

Discussion

Leaderboard

Rules

Team

Submissions

Submission and Description

Public Score

✓

123103_submission_model_forest.csv

Complete · now · Model Forest

0.78229

✓

123103_submission_model_svm.csv

Complete · 9m ago · model_svm

0.77990

✓

123103_submission_knn.csv

Complete · 17m ago · Modelo KNN com minhas features

0.75837

✓

123103_submission_lr_sc_novas_features_4.csv

Complete · 27m ago

0.77511

Conclusão

Diversos ajustes de parâmetros do Random Forest foram realizados para aprimorar o modelo, sendo que o melhor resultado foi enviado ao Kaggle. O ajuste das features, considerando a simetria dos dados dos sobreviventes, contribuiu para a melhoria do desempenho, e a combinação com o Random Forest apresentou o desempenho mais elevado entre os modelos testados.